# A Better Bill Split App Design Doc

## Overview

We will be implementing a bill splitting app where users can upload a picture of their receipt, claim items, and receive the exact amount they are responsible for.

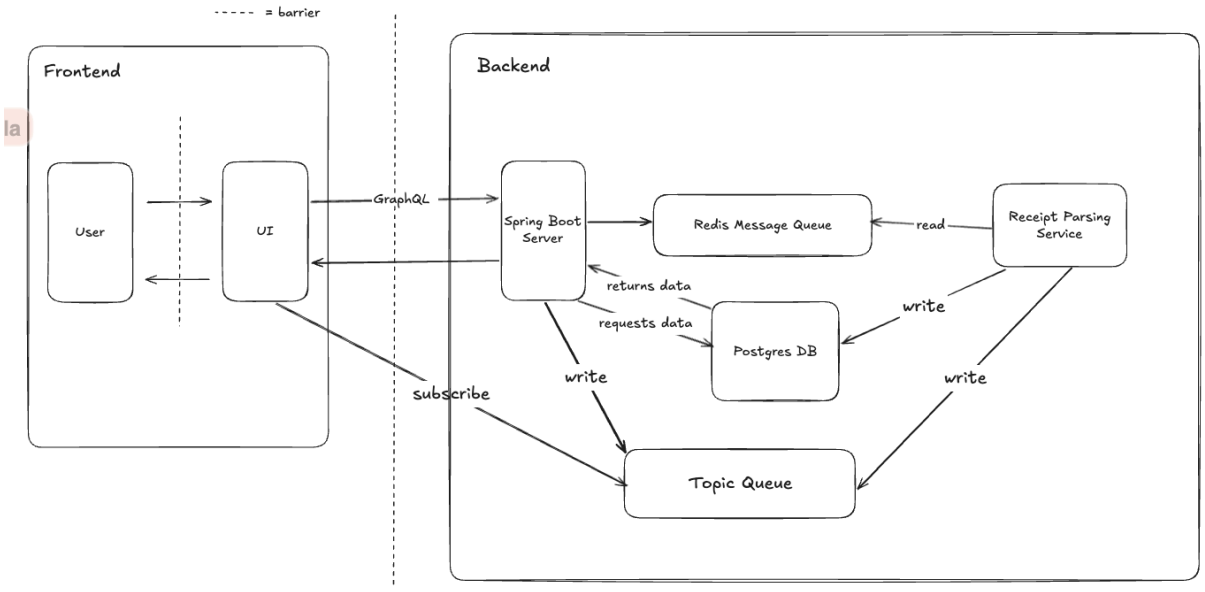
Below are the functionalities of the app:

- The group leader can upload a photo of their receipt.
- Users will see their portion of the bill calculated after claiming items from the receipt.

## Architecture Overview

We will be implementing this project using the following tech stack:

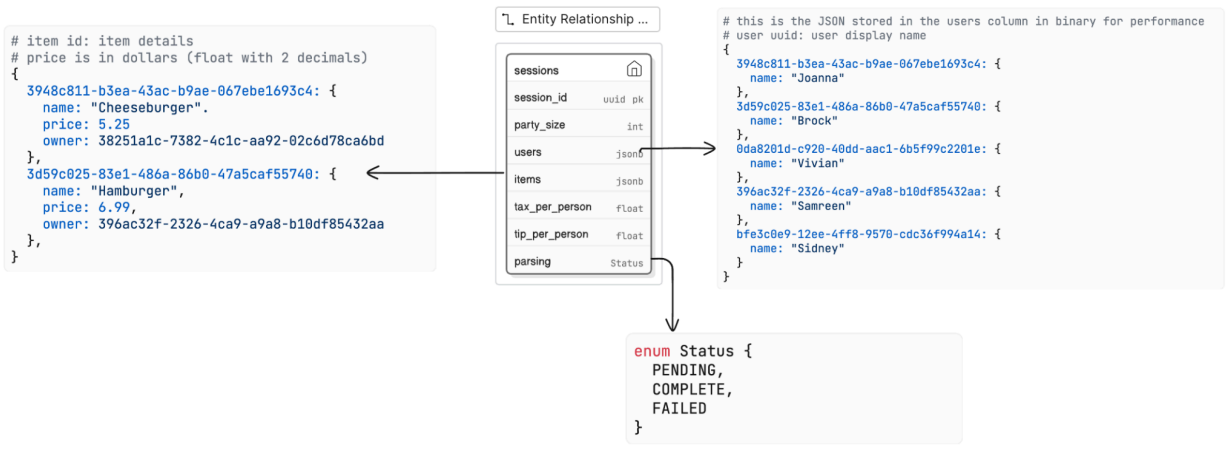
- Frontend: Next.js
- Backend: GraphQL APIs with Spring Boot framework
- Message Queue: Redis
- Database: PostgreSQL with Supabase



Our architecture is designed to handle asynchronous processing, OCR, and real-time concurrency. The pros and cons of our design choices are as follows:

- Frontend: Next.js
    - Pros: It enables a fast, responsive UI for client-side rendering. This allows users a smooth process of claiming and unclaiming items.
    - Cons: Keeping the frontend state consistent across multiple users in real time increases concurrency requirements and adds complexity to the system design, especially when session data changes often.
- Backend: GraphQL APIs with Spring Boot
    - Pros: GraphQL allows the client to request and retrieve the necessary data they need without underfetching or overfetching.
    - Cons: Since the server is stateless, every user interaction, such as a call to claim and unclaim an item, triggers a write to the database. This increases the load on the database and requires using atomic operations when manipulating it to ensure concurrency.

## Database Schema




Our database schema includes a Sessions table for every session that is created. It is initialized with the party size the group leader inputs. To store information about a session efficiently, we will be using a document-based database. The Users column stores information about each user in a JSONB data format with their userID and their names. Similarly, the Items column uses

JSONB to map each item's itemID to its name, price, and owner. In order to ensure there is communication between boundaries, we added a Parsing column to store the parsing status of the receipt at any given time. After the receipt has been sent to the Parsing Service, it will be marked as PENDING. After the Parsing Service sends it to the Topic Queue, it will mark it as COMPELTED. If there are any errors, it will be labelled FAILED. This makes sure that the server and UI remain clear about what the receipt's current status is.

## State Model

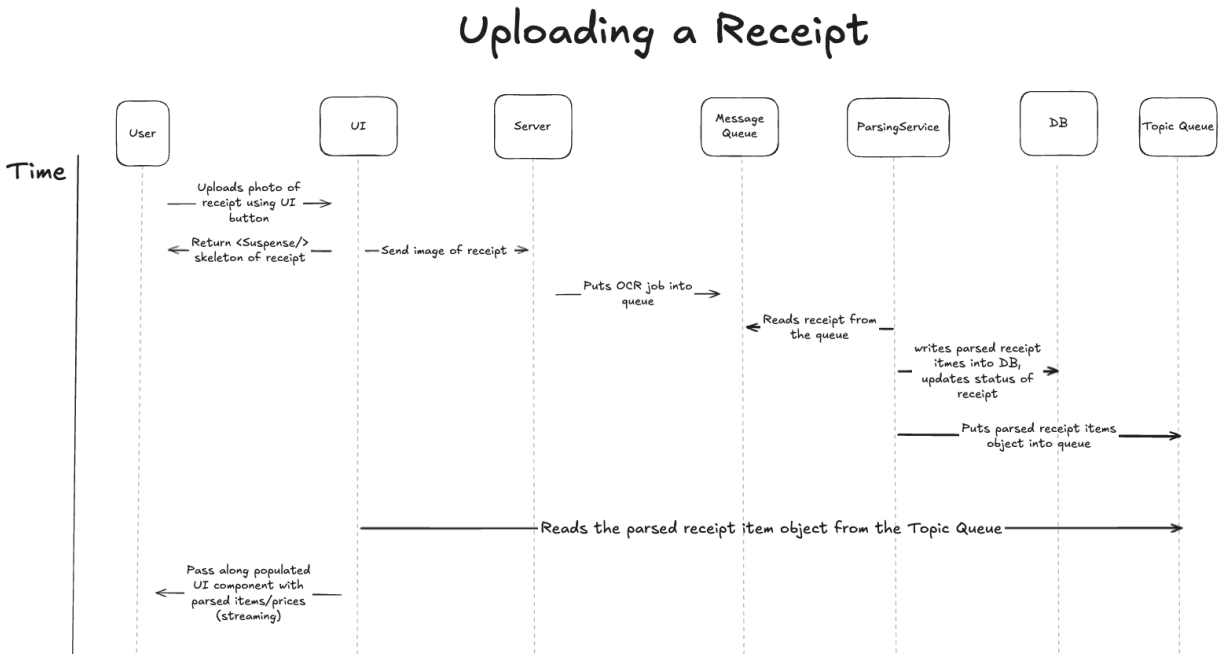
| UI | Server | Message Queue | Topic Queue | Database |
|---|---|---|---|---|
| - state of any checkbox (checked or unchecked)<br>- basic computation of total for each user (one big query to get all info about session e.g. # users, taxes, who's paying for what, etc.) | stateless | - the record of the receipts that are sent to be scanned/parsed<br>- also the corresponding party size for each receipt record | - record of parsed receipts (items) | Session id (same idea as receipt id)<br><br>For each session id:<br>- tip per person<br>- tax per person<br>- party size<br>- users<br>  - user id mapped to user display name<br>- items<br>  - item id mapped to item name, price, and owner |

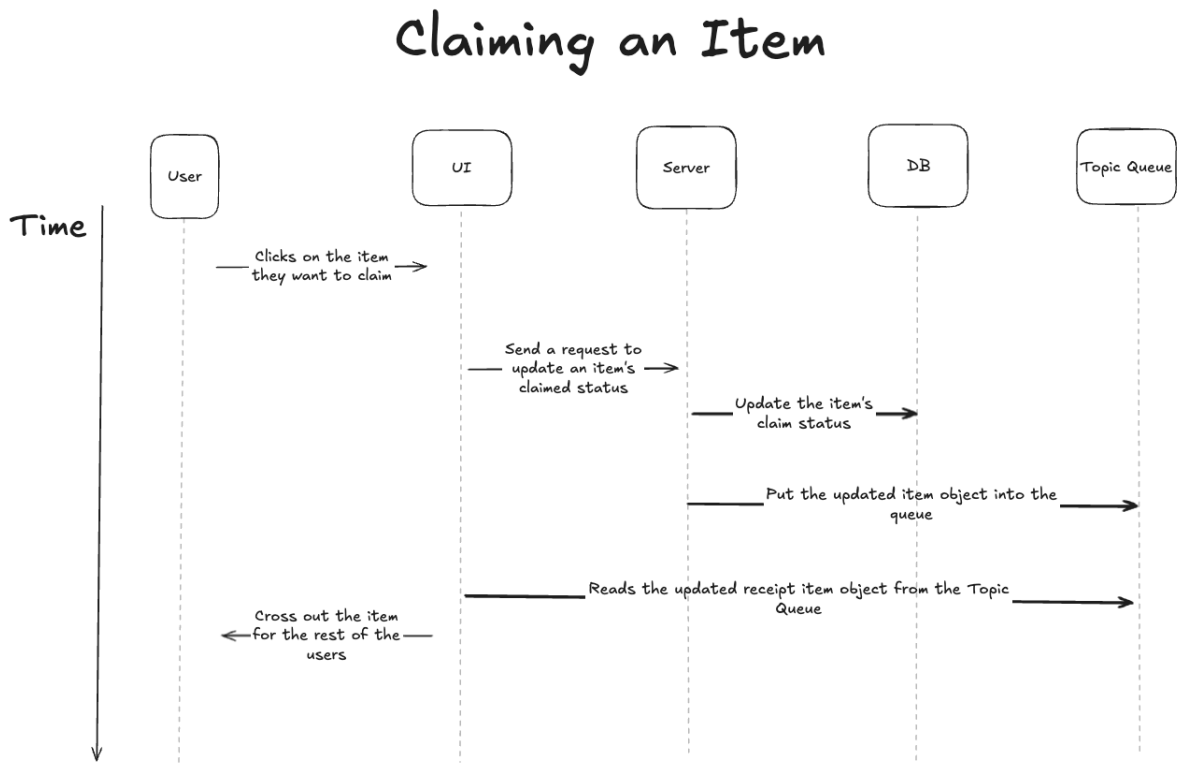
Lifecycle of a receipt:

1. The lifecycle of a receipt begins when a group leader uploads an image of the receipt. This action creates a session record in the database, generates a unique session_id, and sets the parsing_status to pending.
2. The receipt is moved to the message queue for processing. At this time, the UI displays a parsing state. While items aren't yet visible for claiming at this step, other users can join the session via the QR code.
3. Once the receipt parsing service confirms a success status, the itemized data is committed to the items JSONB column in the database. After the database writes the topic queue broadcasts the new data to the UI.
4. Multiple users can concurrently claim or unclaim their items. This updates the owner mapping in the database in real-time. Since the server is stateless, the database acts as the source of truth to prevent multiple users from claiming the same items.

5. The lifecycle ends when the group leader triggers a close session event. The session state becomes read-only, and the bill summary is generated for all participants.

## Sequence Diagrams



## API Design

### Uploading a Receipt

*Problem:* Receipt images are raw binary data. Sending them through a standard GraphQL mutation requires Base64 encoding, which can increase payload size and complexity by a wide margin.

*Solution:* We will utilize Supabase Storage (REST) for a high-performance binary upload, and a GraphQL mutation to trigger the business logic

### Fetch group, bill, user, and other nested data

*Problem:* The session data is highly relational (Session → Items → Users). Using REST format would require multiple round-trips (N+1) to fetch the bill, then users, and specific item claims/

*Solution:* Use a GraphQL query (getSession). This allows the UI (especially for late joiners) to fetch the entire 'Source of Truth,' which includes the session metadata and full users & items JSONB blocks, in a single request.

### Real-time Updates

*Problem:* To satisfy the requirements of UI streaming and immediate updates, the system cannot rely on manual refreshes or HTTP polling, which would overwhelm the database.

*Solution:* Use GraphQL Subscriptions via WebSockets. The server pushes itemParsed events to the message queue for the ReceiptParser worker to work on. As the worker finishes these jobs, the items get streamed to the UI. Additionally, itemStateChange events are handled via the WebSocket whenever an item mutation succeeds, and this ensures all group members stay synchronized without additional network overhead or needing to refresh the page.

### Non-Goals

- Users cannot edit item information or item price (fixed)
- Users can't delete items, but they can UNCLAIM items
- There is also no "creation" involved aside from uploading a receipt to implicitly start/create a session
- The only "update" that's happening on the user end is the claiming/unclaiming items

- Accessing the device camera

**GraphQL Types & Mutations**

```
# types
type Session {
 session_id: ID!
 party_size: Int!
 users: JSONB!          # map of User UUIDs to names
 items: JSONB!          # map of Item UUIDs to details (price, owner)
 tip_per_person: Float
 tax_per_person: Float
 parsing_status: ParsingStatus!
}

type Item {
 id: ID!
 name: String!
 price: Float!
 owner_id: ID           # null if unclaimed
}

enum ParsingStatus {
 INITIALIZING            # job is being submitted to worker
 PARSING                 # worker is parsing receipt
 ACTIVE                  # receipt parsed, streaming items to UI
 CLOSED                  # receipt / session done
 FAILURE                  # failed to parse the receipt
}

type ClaimResult {
 success: Boolean!
 message: String
 updatedItem: Item
}

# queries
type Query {
 # fetches the full 'Source of Truth' for a session
 # needed for late joiners who need their UI hydrated instantly
 getSession(session_id: ID!): Session

 # calculates a specific user's financial responsibility
 # (sum of claimed items + individual share of tax/tip)
```

```
 getUserTotal(session_id: ID!, user_id: ID!): Float
}

# mutations
type Mutation {
 # starts the session and immediately starts background OCR
 # implicitly creates the session row in Postgres
 uploadReceipt(image: String!, partySize: Int!, leaderName: String!):
Session

 # adds a member to the users JSONB map after joining session
 addUserToSession(session_id: ID!, name: String!): Session

 # atomic operation to claim an item
 # solves race conditions by checking 'owner_id IS NULL'
 claimItem(session_id: ID!, item_id: ID!, user_id: ID!): ClaimResult

 # resets the owner of an item to null
 unclaimItem(session_id: ID!, item_id: ID!): ClaimResult

 # finalizes the bill and locks edits
 closeSession(session_id: ID!): Session
}

# subscriptions
type Subscription {
 # pushes items to the UI as the ParsingService finishes them
 itemParsed(session_id: ID!): Item

 # broadcasts any claim/unclaim action to all users instantly
 itemStateChanged(session_id: ID!): Item

 # notifies the group when the bill moves to 'COMPLETED' or 'CLOSED'
 sessionStatusChanged(session_id: ID!): ParsingStatus
}
```

## Failure Modes

### Slow Network

*Problem:* There is a risk that high network latency could cause the UI to receive notifications of parsed items and allow users to attempt to claim an item before said item has been fully persisted in the database.

*Solution:* The Receipt Parsing Service is designed to perform strictly sequential writes. It first commits the itemized data to the Postgres database, and only then pushes the notification to the Topic Queue. This guarantees that by the time a user sees an item in the UI, it is already stored in the database and ready for interaction.

**Server Error(e.g., Receipt Parser Worker Crashes)**

*Problem:* If the backend parsing worker crashes during processing, the UI may fail to display items without providing any feedback to the user, leading to a stalled session

*Solution:* We will implement a robust state-tracking system for the worker that monitors the parsing status as INITIALIZED, PARSING, COMPLETE, CLOSED, or FAILED. By catching errors and updating the database status accordingly, the UI can detect a FAILED state and immediately inform the user so they can attempt to re-upload the receipt rather than infinitely waiting.

**Data Inconsistency & Race Conditions**

*Problem:* Inconsistencies can occur if two users attempt to claim the same item at the exact same time, or if data is lost while moving between the message queue and the database.

*Solution:* To prevent these issues, all database operations related to claiming items are executed as atomic transactions, ensuring only one user can successfully write to a record. Additionally, our system will be using an acknowledgement-based workflow where the database must confirm a successful write before any corresponding update is broadcast to the Topic Queue, ensuring the UI and database remain in constant synchronization.

- Development Goal: have users be able to successfully interact with database (basically excluding actual act of parsing for now)
    - Focus on CRUD app that saves data to database from the perspective of a single group leader:2
        - C - create a session as a group leader
            - Set up UI for what session starting would look like
            - Placeholder button called “receipt” that is a placeholder for the parsing

                - For example, still request that a group leader puts in their display name and party size at the very least (but not ask for the receipt upload just yet)
                    - Submitting this form also creates a session ID in the database
            - R - be able to actually fetch/display all the “parsed” item data
            - U - be able to claim/unclaim items successfully
            - D - maybe we want a button for that that takes them back to a home screen so that we formally know that we can delete that session from our database?
        - Let’s assume that all the parsing has already been done, so they get back the same default receipt information
- Next time on level 2 app:
    - simulate generating a link or QR code that other ppl can join (adding multiple ppl and hence concurrency issues)